

ClipScribe

Local ASR Application - Requirements and Iterative Development Plan

Version 2.0 | Portfolio-ready MVP specification

One-sentence definition

ClipScribe is a privacy-first local automatic speech recognition application that converts short-form screen recordings into faithful, timestamped, exportable transcripts using faster-whisper.

Prepared for implementation with Claude Code in VS Code

Primary use case: iPhone .MOV screen recordings of spoken advice, interviews, educational clips, and news commentary

1. Product Story and Problem Statement

Origin. The project began from a repeated personal workflow: useful ideas often appear in short-form videos, but the knowledge is trapped in audio. The previous workaround required playing a video on one phone while Apple Notes transcribed it on another. That process was slow, sometimes missed the beginning, occasionally failed to detect speech, and often required replaying the clip.

Product opportunity. ClipScribe removes that friction by accepting the original screen recording, extracting its audio, running local ASR, and producing a reviewable transcript. The transcript becomes a reusable source artifact for later analysis, note-taking, interview preparation, fact-checking, or research.

Why this belongs in a portfolio

The project demonstrates problem discovery, scope discipline, local model inference, media processing, quality controls, Streamlit UX, test design, privacy considerations, and a clear evolution path from a focused MVP to a broader personal knowledge workflow.

Representative user story

As a curious learner, I want to upload an iPhone screen recording of a useful TikTok or other short-form video and receive a faithful transcript, so I can preserve the spoken content and use it as the starting point for deeper research without replaying the video into a second phone.

Reference acceptance example

The initial evaluation set should include the recovered “Tell me about yourself” corporate-advice clip. It is valuable because it contains repeated ideas, natural speech, likely filler words, and wording that benefits from both a raw transcript and a separate readable view. The application must preserve the raw output and never silently replace it with an LLM rewrite.

2. Correct Technical Classification

Term	Applies?	Rationale
Automatic speech recognition (ASR) application	Yes	The core model converts spoken audio into text.
Local AI application	Yes	Inference runs on the user’s hardware without a hosted API requirement.
Media-processing and inference pipeline	Yes	The system validates media, decodes audio, runs ASR, applies quality checks, and exports artifacts.

LLM application	No for MVP	Whisper is a specialized ASR model, not a general-purpose LLM workflow.
RAG system	No for MVP	The MVP retrieves no external knowledge before generation.
Agent or agentic workflow	No for MVP	Application code controls the steps; the model does not autonomously choose tools or plan.
Multimodal LLM	No	The video is treated as a container for audio; the system does not jointly reason over frames and speech.

Portfolio wording

“A local ASR and media-processing application with a deterministic transcription pipeline and Streamlit interface.”

3. MVP Scope

In scope

- Upload short video or audio files, with priority support for iPhone .MOV screen recordings.
- Validate file type, size, and readable media streams.
- Decode the embedded audio locally.
- Run faster-whisper inference locally with no API key.
- Display a timestamped raw transcript.
- Expose quality diagnostics and low-confidence warnings without inventing missing words.
- Allow download as TXT, JSON, SRT, and VTT.
- Delete temporary uploaded files after processing.
- Provide automated tests and a small repeatable evaluation set.

Out of scope for MVP

- RAG over a transcript collection.
- Autonomous web or scholarly research.
- LLM-based summarization as a required step.
- Speaker diarization unless added as an optional later milestone.
- Cloud authentication, accounts, payments, or shared storage.

- Claims that hallucinations are eliminated.

4. End-to-End Architecture

Streamlit upload

- > file validation
- > randomized temporary storage
- > media probe / audio decode
- > voice activity detection
- > faster-whisper ASR inference
- > segment normalization and diagnostics
- > raw timestamped transcript
- > TXT / JSON / SRT / VTT export
- > secure cleanup

Layer	Responsibility
Presentation	Streamlit upload, settings, progress, transcript review, downloads.
Application	Coordinates jobs, validates state, handles errors, and preserves separation between raw and derived outputs.
Media	Inspects the container, extracts/decodes audio, normalizes sample rate when needed.
Inference	Loads faster-whisper and performs ASR using a configurable model.
Quality control	VAD, repetition detection, no-speech checks, segment confidence diagnostics, warnings.
Output	Serializes transcript segments into text and subtitle formats.
Privacy	Randomized temp paths, cleanup, no raw transcript logging by default.

5. Functional Requirements

ID	Requirement
FR-01	The UI shall accept .mov, .mp4, .m4a, .mp3, and .wav files.
FR-02	The system shall reject unsupported, empty,

	corrupt, or excessively large files with a clear message.
FR-03	The system shall support direct transcription of iPhone .MOV screen recordings when the decoder can read the embedded audio stream.
FR-04	The user shall be able to choose model size, language auto-detection or override, and optional VAD.
FR-05	The system shall preserve segment start/end timestamps and text.
FR-06	The system shall present a raw transcript as the source-of-truth output.
FR-07	Any cleaned transcript shall be visually and structurally separated from the raw transcript and labeled as derived.
FR-08	The system shall export TXT, JSON, SRT, and VTT.
FR-09	The system shall surface warnings for low-confidence, repeated, or no-speech segments.
FR-10	The system shall clean up temporary media after success or failure.

6. Non-Functional Requirements

ID	Requirement
NFR-01 Accuracy	Optimize for faithful transcription. Never use an LLM to guess inaudible speech.
NFR-02 Privacy	Default mode sends no media or transcript to an external service.
NFR-03 Reproducibility	Pin dependencies and document model/version/hardware used in evaluations.
NFR-04 Usability	A first-time user can upload, transcribe, review, and download without command-line steps after installation.
NFR-05 Reliability	Errors are explicit and temporary files are

	cleaned even when processing fails.
NFR-06 Maintainability	Use typed modular Python with tests around deterministic logic and exporters.
NFR-07 Observability	Log technical events and timings, but do not log raw transcript content by default.

7. Accuracy and Hallucination-Mitigation Strategy

Whisper can emit plausible text during silence, music, or unclear audio. The project shall describe this as a risk to mitigate and evaluate, not a problem that is fully solved.

- Enable voice activity detection to reduce transcription of silence and non-speech regions.
- Use no-speech probability and segment quality signals when available.
- Detect repeated phrases or looping output and flag the affected region.
- Preserve timestamps so the user can replay uncertain segments.
- Allow model-size comparison on the same clip.
- Keep temperature conservative for deterministic transcription.
- Do not let a downstream LLM silently fill missing or unclear content.
- Maintain a small golden set with manually reviewed reference transcripts.

Trust principle

The system should prefer an explicit “[uncertain]” warning over confident invented text.

8. Core Data Contract

TranscriptResult

- source_filename: str
- media_duration_seconds: float | None
- detected_language: str | None
- language_probability: float | None
- model_name: str
- processing_seconds: float
- segments: list[TranscriptSegment]
- warnings: list[str]

TranscriptSegment

- index: int
- start_seconds: float
- end_seconds: float
- text: str

- avg_logprob: float | None
- no_speech_prob: float | None
- warning_codes: list[str]

9. Iterative Software Development Plan

Use deliberate milestones rather than asking Claude Code to generate the entire application at once. Each milestone should end with working code, tests, and a manual verification step.

Milestone	Goal	Exit criteria
M0 - Repository foundation	Create structure, dependency management, linting, typing, test config, and sample fixtures.	Install succeeds; lint/type/test commands run; no application features yet.
M1 - CLI transcription spike	Prove that a real iPhone .MOV can be decoded and transcribed locally.	CLI generates a timestamped TXT/JSON transcript from both reference videos.
M2 - Core service layer	Introduce typed media, ASR, diagnostics, and exporter modules.	Unit tests cover validation, timestamps, warnings, and subtitle formatting.
M3 - Streamlit MVP	Add upload, settings, progress, transcript display, and downloads.	User completes the full workflow from the browser.
M4 - Trust and evaluation	Build golden-set workflow, compare models, measure runtime and observed errors.	README contains a reproducible evaluation table and known limitations.
M5 - Portfolio polish	Architecture diagram, demo video, screenshots, privacy statement, changelog.	Repository is understandable and demonstrable without verbal explanation.
M6 - Optional research handoff	Export structured transcript plus prompts for manual LLM analysis.	Still no claim that the app itself is RAG or agentic.

10. Evaluation Plan

The first two real fixtures should be the three-minute advice video that triggered the project and the recovered corporate “Tell me about yourself” clip. Add synthetic or openly licensed clips for automated CI when redistribution rights are uncertain.

Test case	What it evaluates
Corporate advice clip	Natural speech, repetition, phrasing, punctuation, and raw-vs-readable output.
Three-minute psychology/advice clip	Primary personal use case and longer continuous speech.
Background music clip	Hallucination resistance and VAD behavior.
Fast speech / accent	Model-size tradeoffs and word substitutions.
Long silence	No-speech handling and false text generation.
Corrupt/no-audio media	Clear errors and cleanup.

Recommended metrics: processing time, real-time factor, manual error count, word error rate when a clean reference exists, repeated-phrase count, flagged uncertain segments, and user correction count.

11. Recommended Repository Structure

```
clipscribe/
├─ app.py
├─ CLAUDE.md
├─ README.md
├─ pyproject.toml
├─ docs/
│   └─ ClipScribe_Project_Plan_v2.pdf
├─ src/clipscribe/
│   ├── __init__.py
│   ├── config.py
│   ├── models.py
│   ├── media.py
│   ├── transcriber.py
│   ├── diagnostics.py
│   ├── exporters.py
│   ├── tempfiles.py
│   └─ service.py
├─ tests/
│   ├── unit/
│   ├── integration/
│   └─ fixtures/
├─ samples/           # gitignored personal media
└─ artifacts/         # gitignored generated transcripts
```


12. How to Use Claude Code

1. Place CLAUDE.md in the repository root and this PDF in docs/.
2. Open the repository root in VS Code and start Claude Code there.
3. Begin with a planning-only prompt. Require Claude Code to inspect the environment and wait before broad edits.
4. Approve one milestone at a time.
5. Require exact commands and results after every milestone.
6. Keep the two personal screen recordings outside Git or under a gitignored samples/ folder.
7. Do not give Claude Code permission to add cloud APIs, RAG, or agents unless the MVP is already complete and you explicitly request an extension.

Initial Claude Code prompt

Read CLAUDE.md and docs/ClipScribe_Project_Plan_v2.pdf. Inspect the repository and local environment. Do not write application code yet. Produce your understanding, proposed file structure, dependencies, milestone sequence, risks, and exact verification commands. Call out anything technically unsound and wait for approval before M0.

13. Definition of Done for the MVP

- A user can launch the application with `streamlit run app.py`.
- A real iPhone .MOV screen recording with audio can be uploaded and transcribed locally.
- The raw timestamped transcript is displayed and exportable as TXT, JSON, SRT, and VTT.
- Uncertain or suspicious segments are flagged without invented corrections.
- No API key is required for local transcription.
- Uploaded media and temporary artifacts are cleaned up.
- Automated tests pass and the README documents exact setup and verification steps.
- The repository accurately calls the system an ASR application, not RAG or an agent.
- A short demo shows the original problem, upload flow, transcript, and export.

14. Optional Roadmap After MVP

Phase	Correct terminology	Capability
A	LLM-assisted analysis	User explicitly submits a transcript for claim extraction, summary, or script generation.
B	RAG over a transcript library	Index many transcripts and retrieve relevant timestamped passages for cited Q&A.

C	Tool-using research workflow	Search reputable articles or scholarly sources for claims extracted from a transcript.
D	Agentic research assistant	A model plans and iteratively selects research tools, compares evidence, and produces a cited report.

Scope discipline

These are future extensions. The MVP should remain valuable, complete, and honestly described without them.